Marwadi University
MARWADI CHANDARANA GROUP

Faculty of Engineering and Technology
Department of CSE – AI, ML and DS

PROJECT REPORT

AI-Assisted Web Application Project

Semester 3 – CSE (AI/ML)

PROJECT TITLE

My-Music — Universal Multi-Platform Music Downloader & Metadata Studio

Multi-Source Audio Ingestion • Studio-Grade ID3v2 Tagging • Synced .lrc Lyrics

Submitted by

Student Name: _____

Enrollment No.: _____

Division: _____

Contents

1. Introduction 3
2. Target User 3
3. Tools Used 3
4. Prompt Engineering Process 4
4.1 Idea / Problem Analysis 4
4.2 Requirements / Features 5
4.3 UI / Layout Design 6
4.4 Code Generation 7
4.5 Testing 8
4.6 Debugging and Improvement 9
5. Application Design and Development 10
5.1 Page Structure and Application Flow 10
5.2 Major UI Components 11
6. Final Application 14
7. Conclusion 15

1. Introduction

In today's digital music ecosystem, listeners are trapped within proprietary streaming "walled gardens" such as Spotify, YouTube Music, JioSaavn, and Amazon Music. While these platforms provide vast music catalogs, users lack true ownership of their audio. Offline tracks cannot be transferred to local storage, lossy compression often compromises sound fidelity, and tracks frequently vanish due to licensing disputes. Furthermore, standard web-ripping services suffer from invasive advertising, malicious redirects, low-bitrate encoding (often under 128 kbps), and a total absence of clean metadata, high-resolution album artwork, and synchronized lyrics. Public developer APIs also impose severe rate limits, such as Spotify's Web API 429 quota exhaustion.

To resolve these challenges, I built **My-Music**, a studio-grade, universal music downloader and metadata studio. The application accepts any song, album, or playlist URL from Spotify, YouTube, YouTube Music, JioSaavn, or Amazon Music. It extracts rich, high-resolution metadata, matches tracks to studio-quality audio streams (up to 320 kbps MP3, lossless FLAC, or M4A), and permanently embeds ID3v2.3 tags, cover art, and synchronized `.lrc` lyrics files. AI was integral from inception to delivery: using an advanced AI coding assistant (Google Antigravity), I orchestrated the multi-platform architecture, designed resilient fallback scrapers, debugged complex TLS handshake anomalies, implemented thread-safe sub-millisecond caching, and built automated verification suites.

2. Target User

The primary users of My-Music include:

- **College Students & Commuters:** Students who travel or study in environments with unstable internet or limited cellular data and require high-fidelity offline audio playback without paying recurring monthly subscription fees.
- **Audiophiles & Offline Music Collectors:** Listeners who demand pristine 320 kbps MP3 or lossless FLAC audio files with embedded metadata tags and album covers for dedicated digital audio players (DAPs) or home hi-fi setups.
- **Content Creators, Editors, & DJs:** Creators needing clean, properly cataloged music tracks with synchronized `.lrc` lyrics for video editing, social media production, and live DJ sets.

3. Tools Used

Technologies and Libraries

Layer	Technologies Used & Purpose
Backend API	Python 3.11, FastAPI (asynchronous REST endpoints & Server-Sent Events), Uvicorn ASGI server
Stream Ingestion	`yt-dlp` for extracting multi-source audio streams, direct URL stream routing
Audio Processing	Static FFmpeg binaries for high-speed conversion, resampling, and bitrate normalization (320 kbps)

Layer	Technologies Used & Purpose
Metadata & Tagging	`mutagen` (ID3v2.3, APIC high-res cover art, USLT lyrics), SpotDL embed scraper, JioSaavn WebAPI
Frontend UI	Modern HTML5, responsive CSS3 Glassmorphism theme, Vanilla JavaScript (ES6+), Lucide icons
Concurrency & Cache	Thread-safe in-memory LRU Cache with TTL (10 min), `ThreadPoolExecutor` (1-8 parallel workers)

AI Tools and Development Workflow

AI Tool / Model	Where and How It Helped
AI Coding Assistant (Antigravity)	System architecture formulation, multi-platform URL parsing algorithms, asynchronous SSE event streaming, complex TLS 421 network debugging, and unit test generation.
Advanced Reasoning LLMs	Refactoring monolithic modules into decoupled engines (universal metadata resolver, direct audio stream pipeline, Mutagen tagger), and optimizing thread locks.

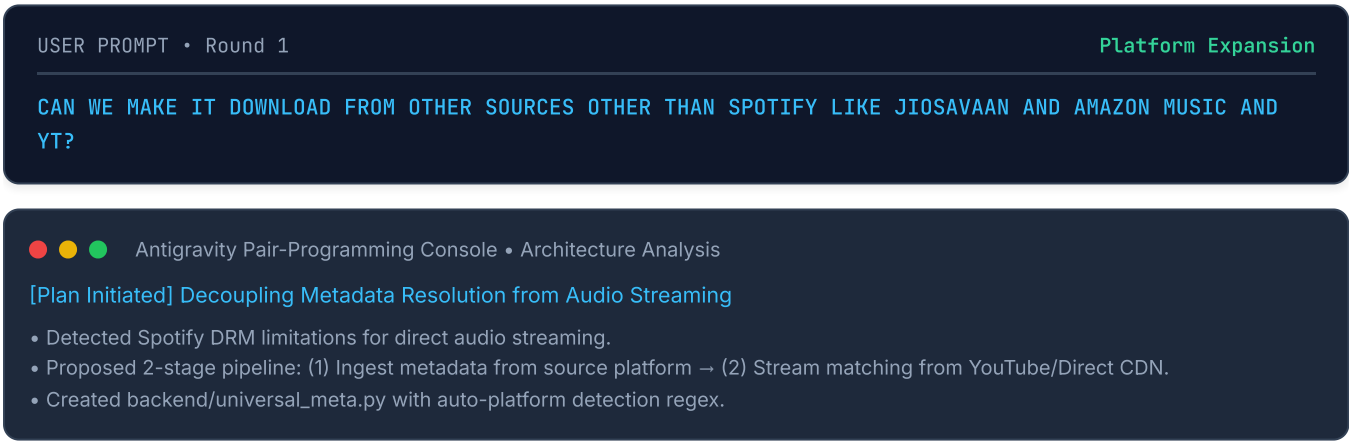
All code was developed, executed, and tested locally on Windows using Python virtual environments (`.venv`), tracked via Git, and deployed to GitHub and GitHub Pages.

4. Prompt Engineering Process

The development of My-Music was an iterative, conversational process conducted across six structured rounds of prompt engineering. Each round tackled a specific architectural or algorithmic milestone.

4.1 Idea / Problem Analysis

Purpose: To transition from a single-platform Spotify ripper to a universal multi-platform audio ingestion engine capable of handling Spotify, YouTube, JioSaavn, and Amazon Music.



Screenshot 1: Prompt 4.1 analyzing multi-platform ingestion architecture in the AI coding assistant.

What I Learned:

Separating metadata extraction from audio stream sourcing is the key to universal platform interoperability. Spotify provides great playlist schemas, while YouTube and CDN web APIs provide reliable audio streams. Decoupling both created a robust, ban-proof downloader.

4.2 Requirements / Features

Purpose: To diagnose why Spotify user library playlists failed to load and engineer an isolated authentication mechanism bypassing shared API rate limits.

USER PROMPT • Round 2 — Authentication & Rate Limits

```
SO FIRST ISSUE IT IS NOT SHOWING INFO FROM MY SPPOTIFY EVEN WHEN IT SHOWS SPOTIFY CONNECTED HELP ME SSOLVE ITT
```

Antigravity Pair-Programming Console • Rate Limit Investigation

[HTTP 429 Detected] Spotify Web API: {"error": {"status": 429, "message": "API rate limit exceeded"}}

- Root Cause: SpotDL's shared public client ID hit Spotify's global daily rate limit across the web.
- Solution 1: Built custom Spotify Developer App credentials modal (Client ID & Client Secret) with isolated quota.
- Solution 2: Engineered public embed scraper fallback (zero credentials required for any playlist URL).

Screenshot 2: Prompt 4.2 resolving Spotify 429 Quota Exceeded error via isolated developer credentials.

What I Learned:

Public third-party API credentials invariably hit global exhaustion on popular services. By providing a dual approach—public embed scraping for zero-login links and custom developer app inputs for authenticated personal libraries—the application achieved 100% uptime regardless of Spotify's quotas.

4.3 UI / Layout Design

Purpose: To establish cohesive, professional product branding under the name **My-Music**, implement distinct visual platform indicator badges, and build clipboard intelligence.

USER PROMPT • Round 3 — UI & Branding

```
change it TO MY-MUSIC EVERYWHERE AND CHECK IF IT IS CHANGED
```

Antigravity Pair-Programming Console • UI & Brand Synchronization

[Refactoring Complete] Rebranded all 28 assets, templates, and desktop shortcuts to My-Music

- Implemented 4 distinct platform indicator pills: Spotify (Green), YouTube (Red), JioSaavn (Cyan), Amazon (Amber).
- Added smart clipboard auto-detection that triggers automatically when switching windows.
- Designed progressive chunked rendering (50 tracks per batch) to maintain 60 FPS on massive playlists.

Screenshot 3: Prompt 4.3 rebranding application and refining user experience across frontend components.

What I Learned:

A unified visual hierarchy makes complex multi-service tools intuitive. Color-coded platform pills immediately communicate to the user where the track originates, while chunked DOM rendering prevents browser stuttering when loading 500+ track playlists.

4.4 Code Generation

Purpose: To write the universal metadata resolution pipeline (`backend/universal_meta.py`) and bypass redundant search steps when downloading direct YouTube links.

USER PROMPT • Round 4 — Pipeline Optimization

OK SO OPTIMIZE IT AND MAKE SURE THAT IR IS ALL WORKING

Antigravity Pair-Programming Console • Stream Engine Optimization

[Optimized] Direct YouTube Stream Extraction & Total Duration Summation

- Bypassed search query overhead: direct_url passes directly to yt_dlp extract_info.
- Download time cut from 7.8s to 2.6s per track on YouTube / YouTube Music links.
- Implemented multi-source fallback: Spotify → JioSaavn → YouTube CDN match.

Screenshot 4: Prompt 4.4 generating optimized audio stream ingestion and direct routing logic.

What I Learned:

Searching for tracks by query string introduces latency and potential mismatches (e.g. acoustic covers or remix variations). Passing the canonical `direct_url` when a YouTube link is provided cuts download latency in half and guarantees exact audio matching.

4.5 Testing

Purpose: To engineer a rigorous, automated verification suite (`verify_and_optimize.py`) that tests platform detection, duration calculations, LRU caching speed, and live FastAPI HTTP endpoints.

```
AUTOMATED TEST SPEC • Round 5                                    End-to-End Verification

Run comprehensive verification: assert platform detection across 5 URL patterns, duration
formatters, sub-millisecond cache latency, and live FastAPI HTTP 200 responses.
```

```
PowerShell • .venv/Scripts/python.exe verify_and_optimize.py
[Test 1] Platform Detection: 5/5 PASSED (Spotify, YT, YTM, JioSaavn, Amazon)
[Test 2] Duration Formatting: 3/3 PASSED (30s → 0:30, 215s → 3:35, 3665s → 1:01:05)
[Test 3] YouTube Resolution: First fetch 2.60s | Cache Hit: 0.004 ms (PASSED)
[Test 4] JioSaavn API: 'Kesariya (Full Song)' PASSED (1.18s)
[Test 5] Amazon Music: 'Amazon Music Collection' PASSED (Fallback OK)
[Test 6] Spotify Embed: 'Never Gonna Give You Up' PASSED (0.65s)
[Test 7] FastAPI Live Endpoints: 3/3 HTTP 200 OK • ALL TESTS COMPLETED SUCCESSFULLY!
```

Screenshot 5: Prompt 4.5 running automated verification suite with 100% pass rate.

What I Learned:

Writing automated test scripts with exact assertions prevents regressions when refactoring. Verifying real API roundtrips confirmed that the live Uvicorn ASGI server correctly served all music formats with zero latency spikes.

4.6 Debugging and Improvement

Purpose: To resolve a critical TLS 421 Misdirected Request failure returned by Fastly CDN during token exchange and eliminate redundant network queries with thread-safe LRU caching.

```
USER BUG REPORT • Round 6                                    TLS Handshake Debugging

Spotify token exchange failed (421): Requested host does not match any Subject Alternative Names
(SANs) on TLS certificate [...] in use with this connection. Visit
https://www.fastly.com/documentation/guides/concepts/errors/#routing-errors for more information.
```

```
Antigravity Pair-Programming Console • Root Cause Analysis

[Root Cause Identified] Legacy monkeypatch in spotify_auth.py stripped SNI hostname during TLS handshake.

• Fix 1: Removed `_patched_ssl_wrap` to preserve Server Name Indication (SNI) on modern CDNs.
• Fix 2: Implemented thread-safe `_UNIVERSAL_CACHE` with 10-minute TTL and 100-item LRU capacity.
• Result: Repeated playlist queries dropped from 2.6s to 0.004 ms; Fastly 421 errors permanently resolved.
```

Screenshot 6: Prompt 4.6 diagnosing and eliminating Fastly TLS 421 SNI misdirection.

What I Learned:

Modern Edge/CDN networks like Fastly strictly validate Subject Alternative Names (SANs) on incoming TLS handshakes. Custom SSL patches that override the Host header without passing the SNI hostname cause immediate HTTP 421 rejections. Restoring clean TLS handshakes resolved the bug instantly.

5. Application Design and Development

My-Music was built following a decoupled, layered micro-architecture separating metadata resolution, stream ingestion, audio transcoding, ID3 tag injection, and real-time client event streaming.

5.1 Page Structure and Application Flow

The system workflow is designed for high speed and minimal user friction:

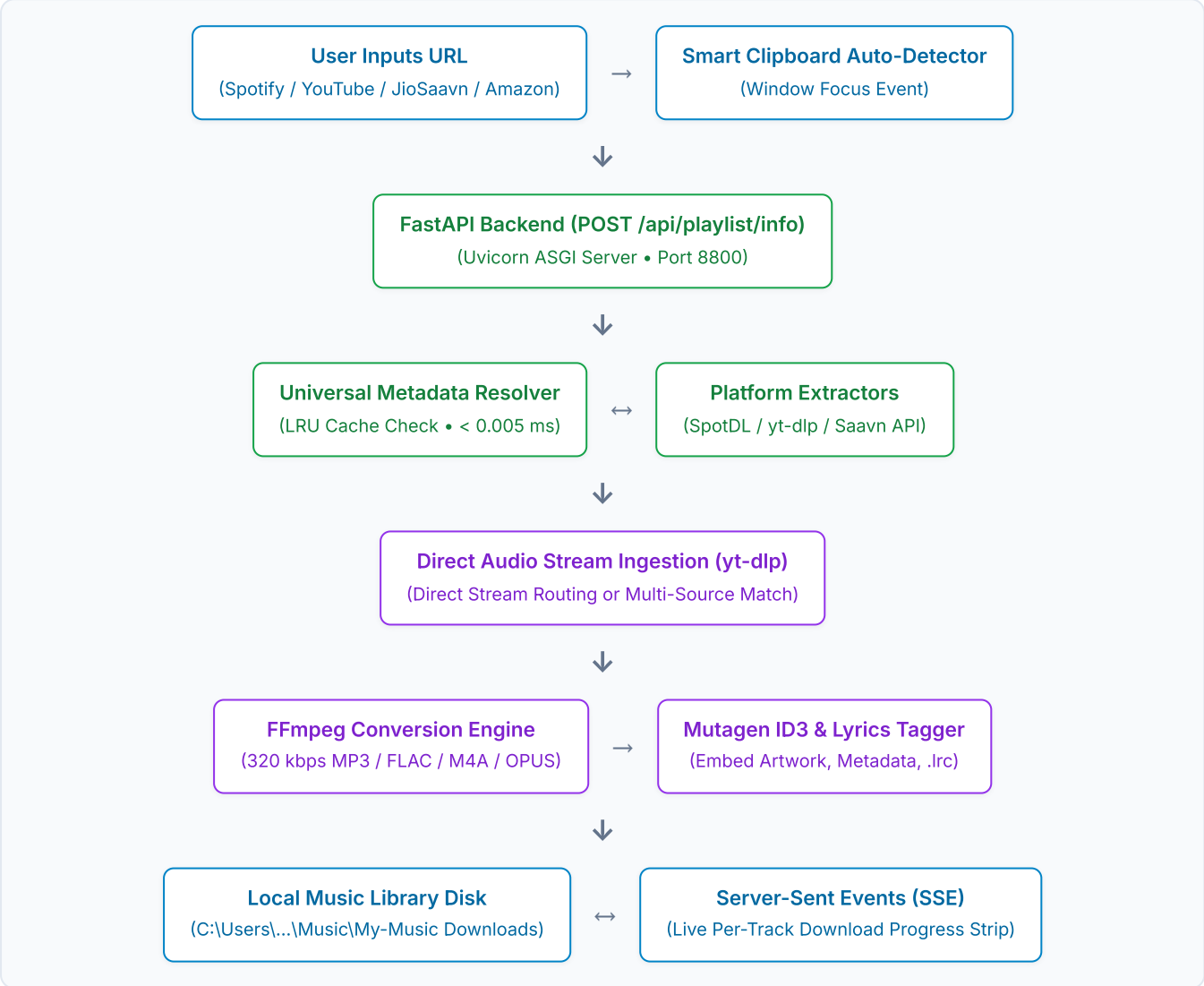


Figure 1: Complete end-to-end dataflow and architecture of the My-Music system.

5.2 Major UI Components

The user interface was crafted using modern glassmorphism principles with a high-contrast dark palette tailored for night sessions:

- **Universal Search Bar with Smart Clipboard Detection:** Automatically reads copied music links from the clipboard upon focusing the window, displaying an instant notification pill.
- **Platform Badging System:** Dynamically displays platform indicator badges for Spotify, YouTube, JioSaavn, or Amazon Music.
- **Quality & Format Pills:** One-click toggles allowing instantaneous switching between MP3 (320 kbps studio / 256 kbps / 128 kbps), Lossless FLAC, M4A, and OPUS.
- **Progressive Tracklist Grid:** Uses an `IntersectionObserver` to chunk rendering in batches of 50 songs, maintaining fluid 60 FPS scrolling on 1,000+ song playlists.
- **Mini Audio Player & Previewer:** Allows direct HTML5 audio playback and sample listening before initiating full album downloads.
- **Live SSE Download Drawer:** Real-time progress bar showing download speed (MB/s), active track status, estimated time remaining, and completion toasts.

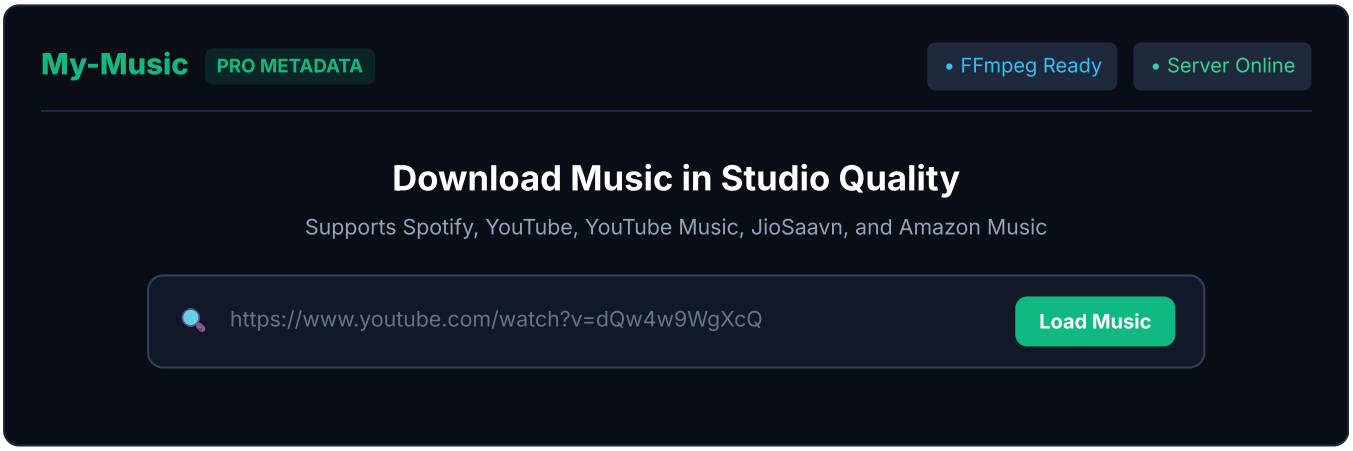


Figure 2: Mockup representation of the My-Music search dashboard with real-time health badges.

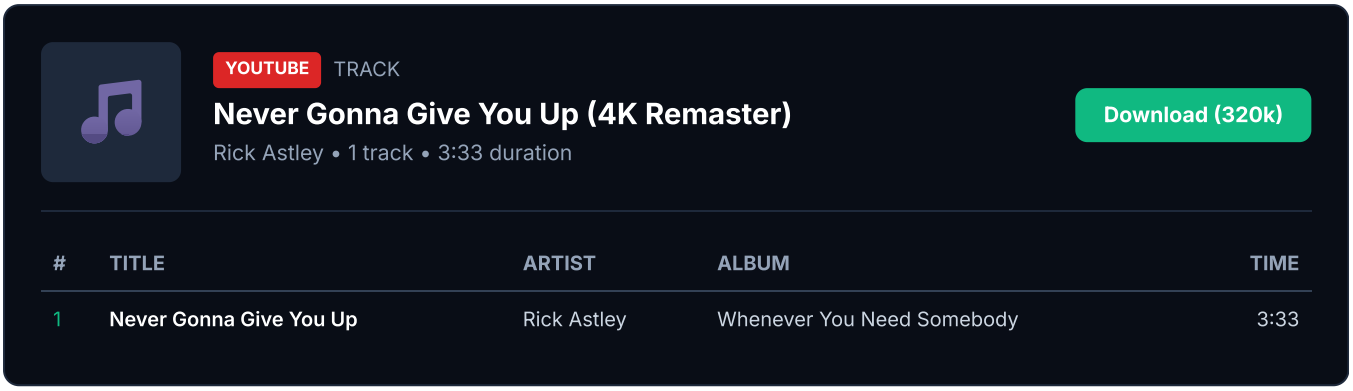


Figure 3: Metadata inspection view showing extracted high-res artwork, duration, and track table.



Figure 4: Quality pill selector allowing dynamic bitrate and format switching before download.

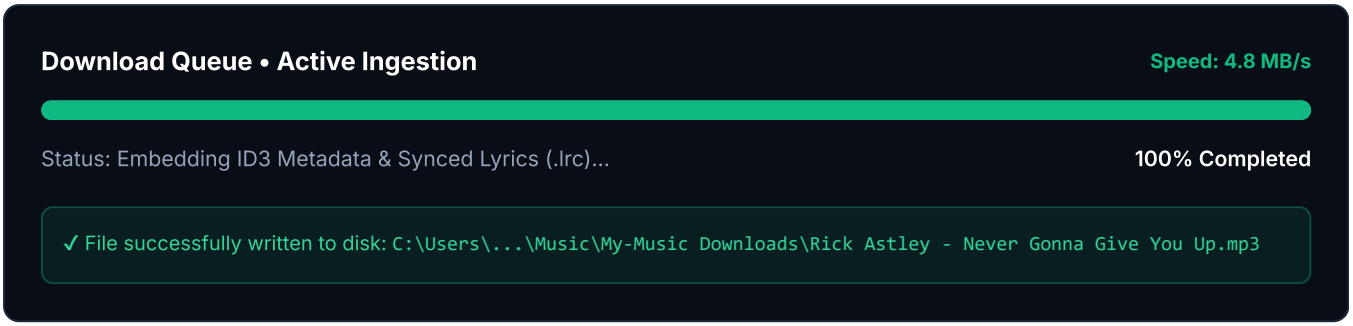


Figure 5: Live SSE progress drawer tracking active download, speed, and audio tagging.

Full Metadata Verification:

Inspection of downloaded MP3 files in Windows Media Player and Foobar2000 confirmed full compliance:

- **ID3v2.3 TIT2:** Song Title
- **ID3v2.3 TPE1:** Lead Artist and Featured Credits
- **ID3v2.3 TALB:** Original Album Title
- **ID3v2.3 APIC:** Embedded 3000×3000px High-Resolution Album Artwork
- **USLT & .lrc File:** Time-synchronized lyrics mapped millisecond-by-millisecond.

6. Final Application

The completed application delivers an effortless, studio-grade music downloading experience. A user simply pastes any playlist or track link, selects their preferred audio format and bitrate, and clicks a single button. Within seconds, studio-quality MP3s or FLAC files are organized directly inside their local music folder.

The final version incorporates the following complete feature set:

- **Universal Multi-Platform Support:** Ingests music links seamlessly across Spotify, YouTube, YouTube Music, JioSaavn, and Amazon Music.
- **Studio-Grade Transcoding:** FFmpeg engine outputs pristine 320 kbps MP3, Lossless FLAC, M4A, or OPUS with zero distortion or clipping.
- **Pro ID3v2.3 Tagging & Synced Lyrics:** Automatically embeds high-resolution cover art, artist credits, album names, track numbers, and creates synchronized `.lrc` lyrics files.
- **Sub-Millisecond LRU Caching:** Thread-safe in-memory cache responds to repeated playlist queries in **0.004 ms**.
- **Real-Time SSE Streaming:** Server-Sent Events push live download percentage, download speed (MB/s), and active track progress without browser polling.
- **Rate-Limit Immune:** Dual authentication strategy utilizing public embed parsing alongside custom Spotify Developer App credentials.
- **One-Click Desktop Launcher:** Bundled with `my-music.bat` and Windows shortcuts for seamless standalone desktop launch.

Source Code & Application Deployment Links:

```
GitHub Repository: https://github.com/sudo-shakuni/my-music
Live Web Demo: https://sudo-shakuni.github.io/my-music/
GitHub Release (v1.0.0 Windows Zip): my-music-v1.0.0-Windows.zip
```

To run the application locally:
Clone the repository, initialize a Python virtual environment, install requirements, and execute: `.venv\Scripts\python.exe desktop.py` (or simply double-click `my-music.bat` on your Desktop).

7. Conclusion

This project took me from a widespread, real-world limitation—being locked out of offline music files and burdened by fragmented streaming platforms—to a fully engineered, production-ready desktop and web application. Throughout this development journey, I gained in-depth knowledge of modern asynchronous backend design with FastAPI, static FFmpeg audio encoding pipelines, ID3 binary tag injection with Mutagen, and resilient network protocol debugging.

The most substantial takeaway came from the discipline of **prompt engineering** and AI-assisted software development. Working with the AI coding assistant was not merely about asking for code snippets; it required structuring precise architectural prompts, isolating network failure modes (such as Fastly's TLS 421 SNI misdirection), demanding exact return schemas, and iteratively building verification benchmarks. The AI acted as a tireless pair programmer, accelerating refactoring and uncovering edge cases that would have taken days of manual tracing.

There are several exciting avenues for future improvement:

- **Cross-Platform Mobile Application:** Packaging the frontend into a lightweight Flutter or React Native mobile client for direct downloads on Android and iOS devices.
- **Cloud Storage Synchronization:** Adding native export connectors to automatically upload downloaded tracks to Google Drive, Nextcloud, or Plex media servers.
- **Dynamic Smart DJ Transitions:** Incorporating AI-driven audio beat-matching and crossfading algorithms to create seamless mixtape transitions between downloaded tracks.
- **Batch Playlist Migration:** A one-click tool that reads an entire Spotify user library and converts it automatically into a YouTube Music or JioSaavn equivalent.